

Desafio Técnico

CaseCellShop

Nível Sênior | Backend

Empresa fictícia. Caso usado exclusivamente para fins de avaliação técnica.

CaseCellShop | Documento do Candidato

Resumo rápido

Formato: perguntas conceituais + mini-tarefa prática backend.

Objetivo: avaliar profundidade em backend, cache, observabilidade, concorrência, resiliência e uso crítico de IA.

Uso de IA: permitido e encorajado. Como sênior, direcione a IA com critério, revise respostas e registre prompts relevantes.

Entrega: respostas conceituais + repositório GitHub público + README + PROMPTS.md, quando aplicável. Priorize uma entrega clara e executável.

Stack preferencial: Node.js + TypeScript no back-end, por aderência ao ambiente técnico esperado.

Se preferir usar outra stack: tudo bem. Explique brevemente a escolha no README e garanta que o projeto seja simples de executar.

Serviços de apoio: use recursos locais/simulados: dados em memória, banco local, Redis/fila local, Docker Compose, LocalStack ou mocks. Datadog é referência conceitual; conta real não é exigida.

Critérios de Avaliação

Importante — Método de Correção:

Serão considerados principalmente:

- análise de causa raiz, riscos e trade-offs;
- cache: TTL, invalidação, fallback e cache stampede;
- observabilidade: logs, métricas, traces, SLO, alertas e runbooks;
- consistência, idempotência e concorrência no checkout;
- resiliência assíncrona: retry, DLQ e reconciliação;
- testes, contrato de API, implementação e uso responsável de IA.

Bem-vindo

Bem-vindo(a) ao desafio técnico sênior da CaseCellShop. O objetivo deste teste é avaliar profundidade técnica, visão sistêmica e maturidade arquitetural. Esperamos que você raciocine sobre garantias do sistema, antecipe modos de falha e proponha estratégias de mitigação com instrumentação adequada.

Contexto do case

Você foi contratado(a) como Desenvolvedor(a) Sênior Backend na CaseCellShop, uma empresa varejista focada na venda de capinhas para celular. A empresa está passando por hipercrecimento: o que antes eram milhares de acessos diários na loja virtual, hoje se transformaram em milhões.

Arquitetura atual

Componente	Descrição
ERP Central	Monolito que gerencia estoque, faturamento, financeiro e contábil. É o coração da empresa.
Loja Virtual	E-commerce que consome produtos, preços e estoque diretamente do ERP via API REST síncrona.
Banco de Dados	ERP utiliza MySQL. Temos acesso de leitura, mas não podemos alterar rotinas, tabelas ou código interno do ERP.
Infraestrutura	Tudo hospedado em datacenter próprio, com intenção de evoluir para serviços mais escaláveis.
Monitoramento	Alertas básicos de performance, com pouca rastreabilidade por pedido, produto ou requisição.

Os 3 problemas identificados

Com o aumento drástico de acessos, três ofensores principais foram identificados:

01 | Performance da vitrine

A vitrine consulta o ERP a cada acesso e fica lenta. O negócio precisa reduzir latência sem exibir preço ou disponibilidade incorretos.

02 | Consistência de estoque

Clientes compram o mesmo item sem estoque. A solução precisa impedir overselling.

03 | Resiliência do checkout

A API do ERP demora para faturar o pedido. A jornada precisa tolerar timeout, retry e processamento assíncrono sem perder rastreabilidade.

Parte 1.A — Perguntas Conceituais

Orientação: responda em texto, tópicos ou diagramas simples. Não precisa escrever código nesta parte.

Pergunta 1 — Diagnóstico, trade-offs e arquitetura alvo

Para cada um dos 3 problemas identificados:

- O que você acredita estar causando o problema na raiz, não apenas o sintoma?
- Qual é o impacto para o cliente, para o negócio e para a operação?
- Compare 2 ou 3 caminhos de solução, considerando custo, complexidade, latência, consistência e esforço operacional.

Inclua uma visão de arquitetura para 30 a 90 dias, citando cache, fila, banco próprio da loja, workers, sincronização, observabilidade e reconciliação com o ERP.

Pergunta 2 — Cache, invalidação e performance da vitrine

Sobre a vitrine de produtos, preços e disponibilidade:

- Onde colocaria cache? Explique o papel de cada camada.
- Descreva TTL, invalidação, cache-aside/refresh-ahead, fallback, prevenção de cache stampede.
- Descreva métricas para validar ganho sem gerar dados obsoletos:
 - Se o cache realmente melhorou performance/custo.
 - Se essa melhora não veio às custas de entregar informação velha ou incorreta.

Pergunta 3 — Observabilidade, Datadog ou equivalente

A liderança quer detectar degradação e furos de estoque antes de reclamações de clientes.

Sem depender de conta real em ferramenta:

- Que logs estruturados você instrumentaria? Quais campos seriam obrigatórios?
- Que métricas criaria usando counters, gauges e histograms para cache, checkout, fila e ERP?
- Que traces distribuídos seriam essenciais nos fluxos GET /products e POST /checkout assíncrono?
- Que SLI/SLO, alertas, dashboard e runbook configuraria no Datadog ou equivalente?

Pergunta 4 — Concorrência, estoque e idempotência

Sobre o furo de estoque no checkout:

- Explique por que uma checagem simples de estoque é insuficiente. Compare atomic update condicional, pessimistic lock, reserva de estoque e distributed lock.
- Como usaria idempotência para tolerar retry, duplo clique e reproprocessamento? Como testaria que a solução evita overselling?

Pergunta 5 — Mensageria, resiliência, contrato e IA

Sobre o checkout assíncrono e a entrega da solução:

- Você publicaria mensagem na fila antes ou depois de gravar o pedido? Como evitaria pedido fantasma e mensagem fantasma? Inclua retry, DLQ, OpenAPI, testes e prompts de IA relevantes.

Parte 1.B — Mini-tarefa Prática Backend

Orientação: suba o código em um repositório público pessoal no GitHub e cole o link no final da sua resposta. Mantenha a solução pequena e executável.

Escopo da mini-tarefa

Implemente um pequeno serviço backend para a CaseCellShop que exponha catálogo de produtos com cache, inicie um checkout assíncrono e permita consultar o status do pedido.

Use dados em memória ou serviços locais/simulados se preferir. O objetivo é demonstrar raciocínio sênior de backend, cache, observabilidade e consistência, não construir um e-commerce completo.

Não é necessário implementar autenticação, pagamento real, deploy, front-end ou integração real com ERP. Explique qualquer simplificação feita por se tratar de desafio técnico.

O que esperamos observar na entrega

Back-end — API, cache e contrato

- ☐ GET /products retorna produtos e usa cache com TTL ou estratégia equivalente.
- ☐ POST /checkout inicia uma compra e retorna 202 Accepted com orderId/status.
- ☐ GET /orders/{orderId}/status permite acompanhar o processamento.
- ☐ Há OpenAPI ou contrato equivalente com schemas de sucesso e erro.

Observabilidade

- ☐ Logs estruturados incluem correlationId/requestId e orderId quando existir.

- ☐ Há métricas relevantes, incluindo cache hit/miss e processamento de checkout/fila.
- ☐ Há trace/span real ou stub justificado ligando request, cache, repo fake e worker.
- ☐ README inclui exemplo de dashboard, alerta ou runbook para Datadog ou equivalente.

Concorrência, assíncrono e entrega

- ☐ Checkout evita venda além do estoque usando atomic update, lock, reserva ou simulação coerente.
- ☐ Há idempotência simples para retries ou duplo clique e worker simulando envio ao ERP.
- ☐ Testes automatizados cobrem regra de negócio, cache e concorrência.
- ☐ README documenta decisões, trade-offs, limitações, instruções para rodar e prompts de IA.

Como entregar

- ☐ Respostas das 5 perguntas conceituais (Parte 1.A).
- ☐ Link do repositório GitHub público (Parte 1.B).
- ☐ README, OpenAPI ou contrato equivalente, e PROMPTS.md quando aplicável.